

Front-End

سلام به همه دوستان

خیلی خوشحالیم که امروز قراره با هم وارد دنیای Front-End بشیم.

قبل از اینکه وارد هر مبحث فنی بشیم، لازمه یک سوءتفاهم خیلی رایج رو از همین اول از بین ببریم:

فرانت‌اند فقط کدنویسی نیست.

اگر فرانت‌اند رو فقط نوشتن (HTML ، CSS یا JavaScript) بدونیم، داریم اصل ماجرا رو کاملاً اشتباه می‌فهمیم.

فرانت‌اند در واقع ترکیبی از چند مهارت مهمه:

فهم رفتار و نیاز کاربر – طراحی تجربه کاربری – پیاده‌سازی رابط گرافیکی – تفکر مهندسی برای حل مسئله

یعنی یک فرانت‌اندکار خوب:

فقط به این فکر نمی‌کنه که کد کار می‌کنه یا نه ؛ بلکه فکر می‌کنه (کاربر چطور با این کد زندگی می‌کنه)

فرض کنید یک سایت خیلی سریع و حرفه‌ای نوشته شده ، اما کاربر:

نمی‌فهمه از کجا باید شروع کنه – دکمه‌ها رو پیدا نمی‌کنه یا وسط کار گیج می‌شه

در این حالت:

از نظر فنی سایت سالمه اما از نظر فرانت‌اند، شکست خورده پس ما در فرانت‌اند با «انسان» طرف هستیم، نه فقط با مرورگر.

```
User
↓
UI / UX Design
↓
HTML → CSS → JavaScript
↓
Framework (React / Vue)
↓
Browser
```

معرفی مسیر کلی Front-End دیاگرام مفهومی :

توضیح خطبه خط دیاگرام (خیلی مهم)

همه چیز از **User** کاربر شروع می شود.

کاربر: عجله دارد _ حوصله ندارد _ اشتباه می کند و انتظار دارد سیستم با او راه بیاید

مرحله بعد **UI / UX Design** است.

در اینجا تصمیم می گیریم:

- چه چیزی کجا باشد؟
- کاربر اول چه چیزی را ببیند؟
- مسیر حرکت او در صفحه چگونه باشد؟

بعد وارد مرحله پیاده سازی می شویم:

HTML برای ساختار صفحه _ **CSS** برای ظاهر و چیدمان _ **JavaScript** برای منطق و تعامل

وقتی پروژه بزرگ می شود:

از **Framework** مثل **React** یا **Vue** استفاده می کنیم تا کدها منظم و قابل توسعه باشند.

در نهایت:

همه چیز داخل **Browser** اجرا می شود و کاربر فقط نتیجه نهایی را می بیند، نه زحمت ما را.

UI / UX Design

ما دقیقاً برای چه کسی و چه کاری چیزی می سازیم؟

اینجاست که **UI** و **UX** وارد می شوند.

اگر **UI** و **UX** درست نباشند: کد تمیز هم بی فایده است _ پروژه شکست می خورد _ کاربر بر نمی گردد

فرانت اند خوب از طراحی شروع می شود، نه از کدنویسی.

بخش اول (User Experience) UX :

UX یعنی: کاربر چه می‌خواهد؟ _ چقدر سریع به هدفش می‌رسد؟ _ در مسیر استفاده چقدر فکر می‌کند؟ _ هرچقدر کاربر کمتر فکر کند، UX بهتر است. فرض کنید کاربر وارد یک سایت می‌شود تا ثبت‌نام کند.

اگر: دکمه ثبت‌نام واضح باشد _ فرم کوتاه باشد _ پیام خطا واضح باشد

کاربر می‌گوید: «اوکی، راحت بود»

این یعنی UX خوب.

اما اگر: دکمه پیدا نشود _ فرم طولانی باشد _ خطا نامفهوم باشد _ کاربر سایت را می‌بندد.

دکمه ثبت‌نام کوچک، خاکستری و گوشه صفحه → بد UX (کاربر نمی‌بیند)

فرم ۱۰ فیلدی بدون توضیح → بد UX (کاربر خسته می‌شود)

پیام خطای مناسب هنگام وارد کردن اطلاعات اشتباه توسط کاربر مثل (رمز عبور باید حداقل ۸ کاراکتر باشد) → خوب UX (کاربر راهنمایی می‌شود)

مفاهیم کلیدی UX

User Flow مسیر حرکت کاربر از شروع تا هدف مثلاً: پرداخت → سبد خرید → محصول → صفحه اصلی

اگر این مسیر پیچیده باشد، کاربر وسط راه سایت را ترک میکنند.

Wireframe طرح خیلی ساده صفحه بدون رنگ و طراحی فقط: جای دکمه _ جای متن _ جای تصویر

Wireframe: یعنی فکر کردن قبل از زیباسازی.

Usability: اینکه استفاده از سیستم چقدر راحت است. نه اینکه فقط «قشنگ» باشد.

Accessibility : طراحی برای همه

نابینا

کم‌بینا

افراد با ناتوانی حرکتی

UX یعنی:

کم کردن بار ذهنی کاربر

کاربر نباید حدس بزند نباید دنبال دکمه بگردد _ نباید فکر کند (الان چی کار کنم؟)

UI (User Interface)

UI یعنی: ظاهر چیزی که کاربر می بیند

برخلاف UX که مربوط به «حس استفاده» است، UI مربوط به «چشم» است.

اجزای اصلی UI شامل این موارد است: هر رنگ پیام دارد (اعتماد، خطر، هیجان) _ فونت بد = خوانایی بد = تجربه بد

فاصله گذاری بد باعث میشود صفحه شلوغ شود و کاربر را فراری می دهد.

نظم بصری چشم کاربر باید بداند کجا را اول ببیند

Design System یعنی: همه دکمه ها شبیه هم باشند _ رنگ ها قانون داشته باشند _ فونت ها ثابت باشند نه اینکه هر صفحه یک سلیقه باشد.

ابزار اصلی UI ابزار استاندارد بازار: Figma

با Figma: طراحی می کنیم _ نمونه اولیه می سازیم _ قبل از کدنویسی اشتباهات را می بینیم.

لینک آموزش مربوط به ui و ux

تفاوت مهم UI و UX

UI = چطور به نظر می رسد

UX = چطور کار می کند

HTML

HTML پایه و اساس تمام وب است . بدون HTML ، چیزی به اسم صفحه وب وجود ندارد.

اما یک نکته خیلی مهم را از همین اول شفاف بگوییم:

HTML زبان برنامه‌نویسی نیست _ HTML زبان ساختاردهی و معنا دادن به محتواست.

HTML مثل اسکلت بدن انسان است.

اسکلت:ظاهر نمی‌دهد _ حرکت نمی‌دهد _ اما همه چیز را نگه می‌دارد

اگر اسکلت نباشد :بدن فرو می‌ریزد

اگر HTML نباشد: صفحه وب بی معنا و بی ساختار است

سلام دنیا برای مرورگر، این فقط یک متن ساده است .هیچ اهمیتی ندارد.

وظیفه اصلی HTML

HTML به مرورگر می‌گوید: این تیتراست _ این منو است _ این محتوای اصلی است _ این فرم است _ این دکمه است

نه اینکه: چه رنگی باشد _ کجا قرار بگیرد _ انیمیشنی داشته باشد

این‌ها وظیفه CSS و JavaScript هستند.

ساختار صفحه یک صفحه استاندارد HTML معمولاً این بخش‌ها را دارد:

(بالای صفحه، لوگو، منو) Header

(محتوای اصلی) Main

Footer (پایین صفحه، اطلاعات تماس، کپی‌رایت)

این یعنی :صفحه باید منطقی ساخته شود، نه تصادفی.

(تگ‌های معنایی) Semantic Tags

تگ‌های معنایی یعنی: تگ‌هایی که اسمشان معنی دارد.

- مثل : footer _ article _ section _ main _ nav _ header
- چرا مهم هستند؟

- گوگل بهتر صفحه را می‌فهمد
- صفحه‌خوان‌ها بهتر کار می‌کنند
- کد تمیزتر و حرفه‌ای‌تر می‌شود

فرم‌ها (Form Elements)

فرم‌ها راه ارتباط کاربر با سیستم هستند.

- کارهایی مثل: ثبت‌نام _ ورود _ جستجو _ ارسال نظر بدون فرم:
- کاربر فقط تماشاگر است
- هیچ تعاملی وجود ندارد

(دسترسی‌پذیری) Accessibility

HTML خوب یعنی: همه بتوانند از سایت استفاده کنند

- مثال‌ها label برای ورودی‌ها _ توضیح متنی برای تصاویر _ ساختار منطقی تیترها
- این کارها کمک می‌کند:
- افراد نابینا
- افراد کم‌بینا
- و ابزارهای کمکی
- بتوانند سایت را درک کنند.

لینک [آموزش](#) html

CSS

تا اینجا یاد گرفتیم:

UI و UX یعنی فکر کردن قبل از کدنویسی

HTML یعنی ساختار دادن به محتوا

حالا می‌رسیم به جایی که بیشتر دانشجوها فکر می‌کنند فرانت‌اند از اینجا شروع می‌شود: CSS

اما یک حقیقت خیلی مهم را همین اول بگوییم:

بدون CSS، صفحه وب فقط یک متن بی‌روح است.

CSS یعنی: شکل دادن _ مرتب کردن _ قابل استفاده کردن صفحه

اگر HTML اسکلت باشد، CSS پوست، لباس و ظاهر بدن است.

نقش واقعی CSS در فرانت‌اند CSS فقط برای زیبایی نیست.

CSS خوب: خوانایی را بهتر می‌کند _ تجربه کاربر را بهتر می‌کند _ استفاده از صفحه را ساده‌تر می‌کند

CSS بد: کاربر را خسته می‌کند _ صفحه را گیج‌کننده می‌کند _ باعث ترک سایت می‌شود

نکته خیلی مهم: بیش از نیمی از کیفیت یک فرانت‌اندکار خوب، به CSS برمی‌گردد.

کسی که CSS را سطحی بلد است:

همیشه با چیدمان مشکل دارد _ صفحه‌هایش ریسپانسیو نیست _ به بن‌بست می‌خورد

هر المان در CSS یک جعبه است.

این جعبه شامل: محتوا _ فاصله داخلی _ حاشیه _ فاصله بیرونی

اگر Box Model کسی را نفهمید:

چیدمان همیشه به هم می‌ریزد

نمی‌دانید چرا المان‌ها جا نمی‌شوند

۲) Layout یعنی : المان ها چطور کنار هم قرار بگیرند.

دو ابزار اصلی : Flexbox برای چیدمان های خطی و ساده _ Grid برای چیدمان های پیچیده و حرفه ای
فرانت اند کار حرفه ای:

می داند کجا Flex استفاده کند

می داند کجا Grid بهتر است

۳) Responsive Design . کاربر فقط با لپ تاپ سایت را نمی بیند.

کاربر: موبایل دارد _ تبلت دارد _ مانیتور بزرگ دارد

CSS باید صفحه را با اندازه صفحه تطبیق دهد.

اگر صفحه روی موبایل خراب باشد: UX نابود می شود

۴) Typography. رنگ فونت و رنگ فقط سلیقه نیست.

فونت بد: چشم را خسته می کند _ متن را ناخوانا می کند

رنگ بد : حس اعتماد را از بین می برد _ کاربر را گیج می کند

۵) Animation :

حس زنده بودن می دهد _ توجه کاربر را هدایت می کند

انیمیشن زیاد: UX را خراب می کند _ آزاردهنده است

لینک [آموزش css](#)

JavaScript

UI و UX را شناختیم

HTML را به عنوان اسکلت یاد گرفتیم

CSS را برای ظاهر و چیدمان بررسی کردیم

اما هنوز یک چیز کم است.

صفحه ما :

قشنگ است و ساختار هم دارد ولی هوشمند نیست.

اینجاست که JavaScript وارد می شود.

JavaScript یعنی :

فکر کردن _ تصمیم گرفتن _ واکنش نشان دادن

اگر HTML اسکلت باشد و CSS ظاهر، JavaScript مغز صفحه است.

JavaScript باعث می شود: دکمه کلیک پذیر شود _ فرم بررسی شود _ محتوا تغییر کند _ داده از سرور گرفته شود

بدون JavaScript: صفحه فقط یک پوستر زیباست _ هیچ تعاملی ندارد

فرض کنید یک دکمه داریم:

بدون JavaScript دکمه فقط دیده می شود

با JavaScript با کلیک روی دکمه، کاری انجام می شود و پیام نمایش داده می شود و داده ارسال می شود

این تفاوت «دیدن» و «تعامل» است.

مفاهیم پایه ی جاوا اسکریپت

Variables (۱)

نگه داشتن داده

مثل: اسم کاربر _ وضعیت لاگین _ تعداد آیتم ها

بدون متغیر: هیچ منطقی وجود ندارد

Functions (۲)

یک کار مشخص

قابل استفاده چندباره

تابع باعث می‌شود: کد تمیزتر شود_ تکرار کمتر شود _ فکر برنامه‌نویسی شکل بگیرد

DOM (۳)

JavaScript بتواند HTML را تغییر دهد

مثلاً: متن عوض شود_ کلاس اضافه شود_ المان حذف شود_ اینجاست که JavaScript با صفحه حرف می‌زند.

Events (۴)

واکنش به کاربر

مثل: کلیک_ تایپ_ اسکرول

فرانت‌اند بدون Event یعنی: صفحه بی‌جان

۵) ارتباط با سرور :

داده بگیرد _ داده بفرستد

مثلاً: لیست محصولات _ اطلاعات کاربر

لینک [آموزش](#) JavaScript

Framework

تا اینجا ما یاد گرفتیم:

برای کاربر طراحی کنیم

ساختار صفحه را با HTML بسازیم

ظاهر و چیدمان را با CSS درست کنیم

منطق و تعامل را با JavaScript اضافه کنیم

در این مرحله، یک سؤال طبیعی پیش می‌آید:

وقتی پروژه بزرگ شد، وقتی کد زیاد شد، وقتی چند نفر هم‌زمان روی پروژه کار کردند، چه کار باید کرد؟

اینجاست که Framework وارد می‌شود.

Framework یعنی:

یک چارچوب آماده برای نظم دادن به کد برای پروژه‌های بزرگ و واقعی

Framework قرار نیست: جای فکر کردن را بگیرد_ جای یادگیری پایه را بگیرد

Framework قرار است: کد را سازمان‌دهی کند_ توسعه را راحت‌تر کند

فرض کنید یک پروژه کوچک دارید: یک صفحه_ چند دکمه _ کمی JavaScript

بدون Framework هم قابل مدیریت است.

اما وقتی پروژه: چندین صفحه دارد _ صدها کامپوننت داردو تیمی توسعه داده می‌شود

بدون Framework: کد شلوغ می‌شود_ نگهداری سخت می‌شود _ باگ زیاد می‌شود

نقش Framework در فرانت‌اند Framework کمک می‌کند:

کد تکه‌تکه و منظم شود

هر بخش مسئولیت مشخص داشته باشد

تغییرات راحت‌تر اعمال شود

Framework یعنی: نظم _ مقیاس‌پذیری _ همکاری تیمی بهتر

معرفی کوتاه‌Framework معروف React: استفاده گسترده در شرکت‌ها _ مبتنی بر کامپوننت

React به شما یاد می‌دهد: صفحه را به قطعات کوچک تقسیم کنید _ فکر مهندسی داشته باشید _ Vue ساده‌تر برای شروع_ یادگیری سریع‌تر_ ساختار خوانا

مناسب کسانی که:

تازه وارد هستند

می‌خواهند سریع نتیجه ببینند

Angularسنگین‌تر

ساختار سازمانی

مناسب پروژه‌های بزرگ شرکتی

نکته بسیار مهم (حتماً گفته شود) Framework را نباید زود شروع کرد:

اگر HTML را نفهمی CSS_ را بلد نباشی JavaScript_ را درک نکرده باشی

لینک [آموزش](#) React